

Yazılım Tasarım Dokümanı (SDD)

DevConnect

Yazılımcılar için Sosyal Medya Platformu

Yazılım Mühendisliği Dersi – BM314

Hazırlayanlar:

Berat Demir – 23118080762

Mehmet Berkay Yalçın – 22118080045

Kelami Batuhan Bölükbaşı – 23118080787

22 Nisan 2026

REVİZYON GEÇMİŞİ

Versiyon	Tarih	Açıklama	Yazar
1.0	22 Nisan 2026	SDD İlk Sürümü	Proje Ekibi

İÇİNDEKİLER

REVİZYON GEÇMİŞİ.....	2
1. KAPSAM.....	5
1.1 Tanım.....	5
1.2 Sisteme Genel Bakış.....	5
1.3 Dokümana Genel Bakış.....	5
2. İLGİLİ DOKÜMANLAR.....	5
3. YKE ÇAPINDA TASARIM KARARLARI.....	6
3.1 Mimari Yaklaşım.....	6
3.2 Girdi ve Çıktılara İlişkin Tasarım Kararları.....	7
3.3 Veritabanı Tasarım Kararları.....	7
3.4 Emniyet, Güvenlik ve Gizlilik Kararları.....	7
3.5 Esneklik, Erişilebilirlik ve İdame Kararları.....	7
4. YKE'NİN YAPISAL TASARIMI.....	8
4.1 YKE Bileşenleri.....	8
4.1.1 WU-01: Auth Modülü.....	9
4.1.2 WU-02: Profil ve Ağ Modülü.....	9
4.1.3 WU-03: Feed ve İçerik Modülü.....	9
4.1.4 WU-04: Medya ve Etkileşim Modülü.....	9
4.1.5 WU-05: Gerçek Zamanlı Mesajlaşma Modülü.....	10
4.1.6 WU-06: Ortak Altyapı.....	10
4.2 Genel Çalıştırma (Execution) Kavramı.....	10
4.2.1 Kullanıcı Girişi Akış Diyagramı.....	10
4.2.2 Gönderi Oluşturma Akış Diyagramı.....	11
4.2.3 Gerçek Zamanlı Mesajlaşma Akış Diyagramı.....	12
4.2.4 Durum Geçiş Diyagramı.....	13
4.3 Arayüz Tasarımı.....	14
4.3.1 IF-6: Express.js REST API Endpoint Tasarımı.....	15
4.3.2 Veritabanı Şema Tasarımı.....	15
5. YKE DETAYLI PLANI.....	17
5.1 WU-01: Auth Modülü Detaylı Planı.....	17
5.2 WU-02: Profil ve Ağ Modülü Detaylı Planı.....	17
5.3 WU-03: Feed ve İçerik Modülü Detaylı Planı.....	18
5.4 WU-04: Medya ve Etkileşim Modülü Detaylı Planı.....	18
5.5 WU-05: Gerçek Zamanlı Mesajlaşma Modülü Detaylı Planı.....	19
5.6 WU-06: Ortak Altyapı Detaylı Planı.....	19
6. GEREKSİNİMLERİN İZLENEBİLİRLİĞİ.....	20
6.1 SRS Gereksinimlerinden Yazılım Birimlerine İzlenebilirlik.....	20
6.2 Yazılım Birimlerinden SRS Gereksinimlerine İzlenebilirlik.....	21
7. NOTLAR.....	21
7.1 Kısaltmalar ve Tanımlar.....	21
7.2 Genel Notlar.....	22
8. EKLER.....	22
Ek A – Teknoloji Yığını Özeti.....	22

1. KAPSAM

1.1 Tanım

Bu belge, DevConnect yazılım uygulamasının tasarım sonuçlarını geliştirme amaçlı sunan bir Yazılım Tasarım Belgesi'dir (SDD). DevConnect sisteminin mimari ve tasarım tercihleri bu belgeden alınmıştır ve BM314 Yazılım Mühendisliği dersimizin bir parçası olarak tasarlıyoruz. DevConnect — kısaca DC olarak adlandırılacak — yazılım geliştiricilerin teknik bilgileri paylaşmasına ve diğer geliştiricilerle ağ kurmasına olanak tanıyan web tabanlı bir sosyal ağ sitesi olarak tasarlanmıştır.

1.2 Sisteme Genel Bakış

DevConnect, Stack Overflow'u LinkedIn gibi profesyonel ağ işlevselliği ile birleştiren, geliştiricilere odaklı bir sosyal medya uygulamasıdır. Uygulama, kullanıcı kimlik doğrulama, profil yönetimi, içerik paylaşımı (metin, resimler ve kod blokları), akış algoritması, medya depolama, beğeni ve yorum etkileşimleri ile gerçek zamanlı özel mesajlaşma (DM) gibi modüllerden oluşmaktadır. Sistem, React/Next.js tabanlı bir ön yüz, Express.js/Node.js tabanlı bir arka uç ve Supabase (PostgreSQL) tabanlı bir veritabanı katmanından oluşmaktadır. Gerçek zamanlı iletişim, Supabase Realtime / WebSocket altyapısı ile sağlanmaktadır. Uygulama Vercel üzerinden dağıtılacaktır.

Kullanıcılar: Sisteme kayıt olan yazılım geliştiriciler.

Geliştirici ekip: Berat Demir, Mehmet Berkay Yalçın, Kelami Batuhan Bölükbaşı.

İlgili dokümanlar: SPMP, SRS

1.3 Dokümana Genel Bakış

Bu belge yedi ana bölümden oluşmaktadır. Bölüm 1 kapsamı tanımlar. İlgili belgeler Bölüm 2'de listelenmiştir. Bölüm 3, bu sistemde yapılan özel tasarım tercihlerini açıklar. Yazılımın yapısal bileşenleri Bölüm 4'te gösterilmiştir. Yapıların planı Bölüm 5'te açıklanmıştır. Özelliklerin izlenebilirliği de ayrıntılı olarak Bölüm 6'da belirtilmiştir. Bölüm 7 notlar ve kısaltmalar sağlar.

2. İLGİLİ DOKÜMANLAR

Bu bölümde, bu SDD'de referans alınan tüm dokümanlar listelenmiştir.

No	Doküman Adı	Tarih	Kaynak
[1]	DevConnect Yazılım Proje Yönetim Planı (SPMP)	11 Mart 2026	Proje Ekibi
[2]	DevConnect Yazılım Gereksinim Belirtimi (SRS)	01 Nisan 2026	Proje Ekibi
[3]	React Resmi Dokümantasyonu	—	https://react.dev/
[4]	Next.js Resmi Dokümantasyonu	—	https://nextjs.org/

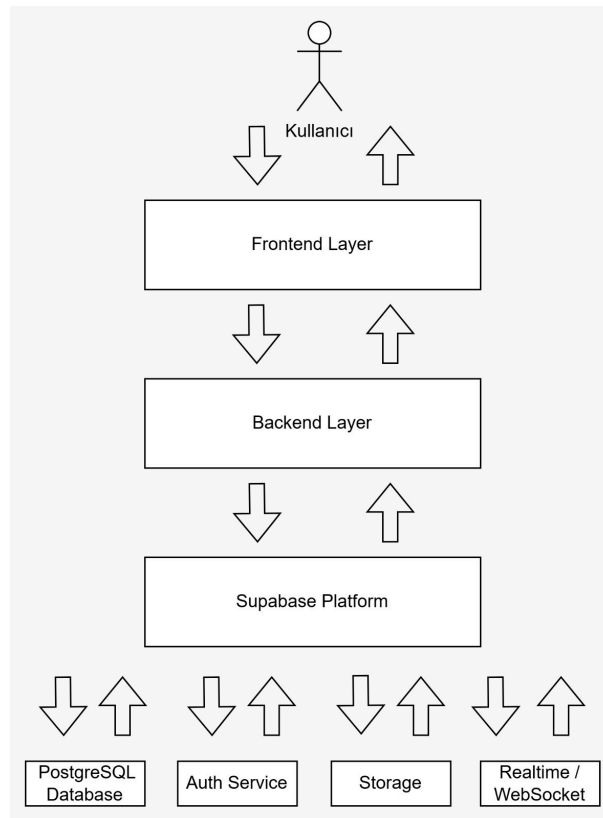
[5]	Supabase Resmi Dokümantasyonu	–	https://supabase.com/
[6]	Socket.IO Resmi Dokümantasyonu	–	https://socket.io/
[7]	PostgreSQL Resmi Dokümantasyonu	–	https://www.postgresql.org/
[8]	Express.js Resmi Dokümantasyonu	–	https://expressjs.com/
[9]	IEEE Std 830 – SRS Şablonu	–	IEEE
[10]	ISO/IEC/IEEE 29148:2018 – Gereksinim Mühendisliği	–	ISO/IEC

3. YKE ÇAPINDA TASARIM KARARLARI

Bu bölüm, DevConnect platformunun her modül ve sisteminin tasarımına dayalı tüm ilgili tasarım kararlarını kapsar.

3.1 Mimari Yaklaşım

DevConnect, katmanlı bir mimari mantığı kullanır. Görsel katman (Frontend), iş mantığı katmanı (Backend API) ve veri katmanı (Veritabanı) birbirinden tamamen bağımsız olarak hazırlanmıştır. Bu şekilde, her katman gerektiğinde bağımsız olarak geliştirilebilir, test edilebilir ve değiştirilebilir.



3.2 Girdi ve Çıktılara İlişkin Tasarım Kararları

Frontend ve backend, tüm kullanıcı girişlerini inceleyecek. Frontendde, React form doğrulaması yapılacak ve sunucu tarafı doğrulama Express ara yazılımı kullanılarak gerçekleştirilecek. Tüm API yanıtları JSON formatında geri gönderilecek. Başarılı işlemler HTTP durum kodları 200/201 kullanacak ve geçersiz istekler 400/401/403/404/500 durum kodlarını kullanacak. backendde resim yüklemeleri için 2 MB sınırı olacak ve dosyalar sıkıştırılarak Supabase Depolama'ya aktarılacak. Metin içeriği için maksimum karakter sayısı 5000 olabilir.

3.3 Veritabanı Tasarım Kararları

Veritabanı, PostgreSQL ile Supabase platformu olarak planlanmaktadır. Tüm tablolar arasında ilişkiler kurmak için yabancı anahtar kısıtlamaları uygulanacak ve veri bütünlüğü veritabanı seviyesinde korunacak. Bu yaklaşım, takipçi sayıları gibi kritik verileri frontende bırakmamakla kalmaz; bunlar Supabase Veritabanı Tetikleyici mekanizmaları aracılığıyla veritabanı seviyesinde otomatik olarak güncellenecek. Bunu yapmanın asıl amacı, yüksek eşzamanlı erişimden kaynaklanabilecek veri tutarsızlığını önlemektir. Mesajlaşma verilerini ölçeklendirirken, MongoDB alternatif bir depolama seçeneği olarak düşünülebilir (SPMP Bölüm 2.3).

3.4 Emniyet, Güvenlik ve Gizlilik Kararları

Sistemde şu güvenlik önlemleri alınmıştır:

- Tüm istemci-sunucu iletişimi HTTPS ve WSS protokolleri üzerinden güvenli bir şekilde sağlanır.
- Kullanıcı kimlik doğrulaması JWT token sistemi ile yapılacaktır. Varsayılan olarak, token süresi 1 saat olacaktır.
- Tüm tablolarda Supabase Satır Düzeyi Güvenlik (RLS) politikaları uygulanacaktır. Sadece erişim hakkınız olan verilere erişebileceksiniz.
- Veritabanında hiçbir şekilde şifre saklanmayacak; Supabase Auth tarafından güvenli hash sistemleri kullanılacaktır.
- API anahtarları ve bağlantı bilgileri çevre değişkenleri (.env) kullanılarak korunacak ve kaynak kodda yer almayacaktır
- SQL Enjeksiyonu ve XSS saldırılarını önlemek için parametrelili sorgular ve giriş temizleme kullanılacaktır.
- Özel mesajlar sadece onlara ait olan kullanıcılar tarafından görülebilecektir.

3.5 Esneklik, Erişilebilirlik ve İdame Kararları

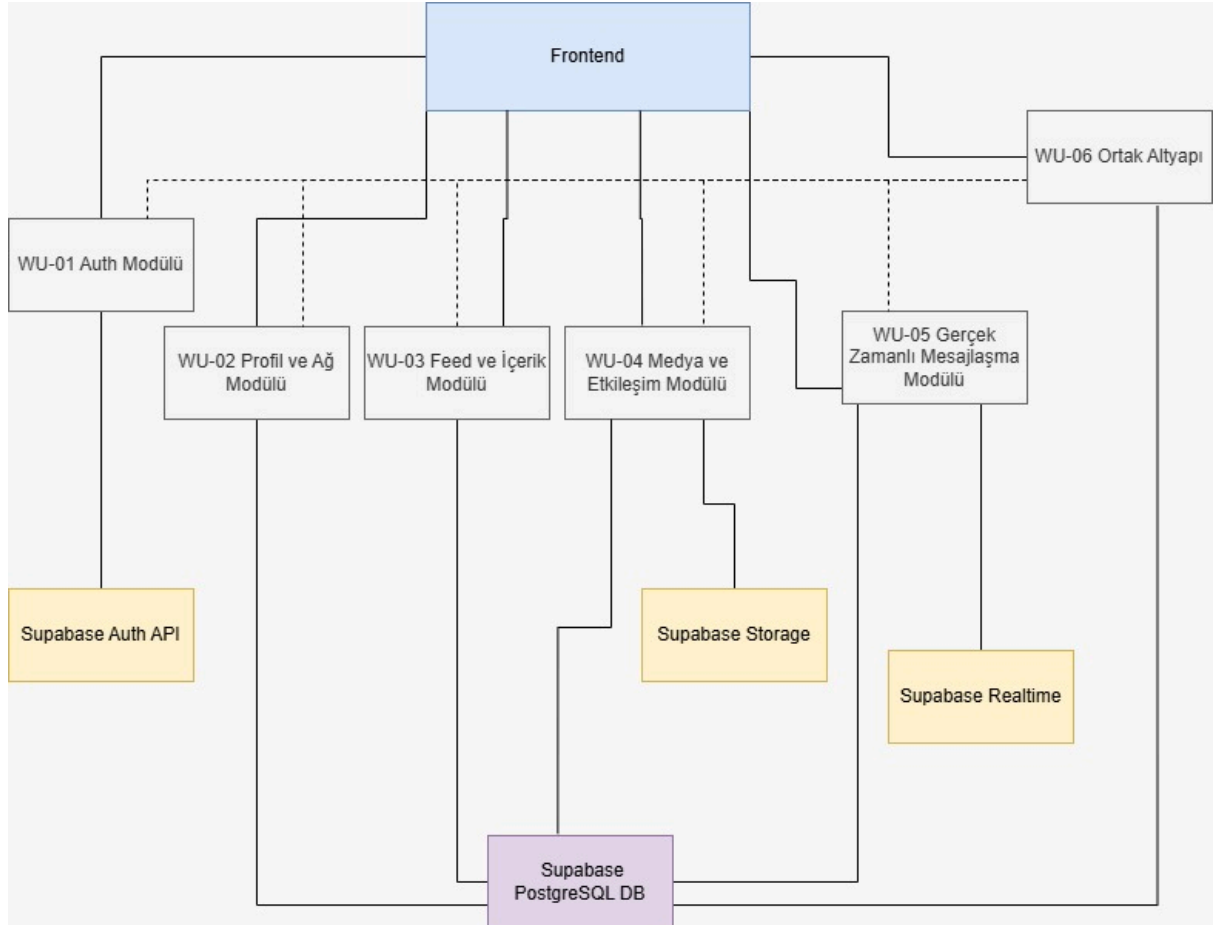
Kod TypeScript dilinde olacak. Bu yaklaşımı seçerek derleme zamanı, tür hatalarını yakalamayı hızlandırır ve hata kontrolünü daha hızlı hale getirir, ayrıca kod okunabilirliğini önemli ölçüde artırır. Modüler bir mimariye sahip olacak. Modüllerin her biri (Auth, Profil, Akış, Medya ve Mesajlaşma) bağımsızdır ve bağımsız olarak geliştirilebilir ve test edilebilir. Vercel'in sunucusuz mimarisi, yazılımın sürekli erişilebilir olacak şekilde tasarlanacağı anlamına gelir. Sistem, WebSocket bağlantısı kaybolduğunda Degradable Moduna geçer — bu, mesajlaşma başarısız olsa bile akış, profil ve diğer hizmetlerin bağlantı olmadan çalışmaya devam edeceği anlamına gelir. Performans: sonsuz kaydırma, veritabanı sorgularında indeksleme ve istemci tarafı görüntü sıkıştırma teknikleri kullanılacaktır.

4. YKE'NIN YAPISAL TASARIMI

4.1 YKE Bileşenleri

DevConnect sistemi altı ana yazılım modülünden (bileşeninden) oluşmaktadır. Her bileşen, SRS'deki fonksiyonel gereksinimlerle doğrudan ilişkilidir.

Bileşen ID	Bileşen Adı	İlgili FR'ler	Durum
WU-01	Auth Modülü – Kimlik Doğrulama	FR-1.1 – FR-1.7	Yeni Tasarım
WU-02	Profil ve Ağ Modülü	FR-2.1 – FR-2.6	Yeni Tasarım
WU-03	Feed ve İçerik Modülü	FR-3.1 – FR-3.5	Yeni Tasarım
WU-04	Medya ve Etkileşim Modülü	FR-4.1 – FR-4.6	Yeni Tasarım
WU-05	Gerçek Zamanlı Mesajlaşma Modülü	FR-5.1 – FR-5.6	Yeni Tasarım
WU-06	Ortak Altyapı (API Gateway, RLS)	IAR-1 – IAR-4	Yeni Tasarım



4.1.1 WU-01: Auth Modülü

Bu modülün amacı, kullanıcıların sisteme kaydolmasını, giriş yapmasını ve güvenli bir şekilde çıkış yapmasını sağlamaktır. Bu bileşen, Supabase Auth hizmeti ile entegre çalışır. Bu modül, JWT'lerin üretilmesi ve doğrulanmasından, şifrelerin hashlenmesinden ve RLS politikalarının uygulanmasından sorumludur. Diğer tüm modüllerin kullanıcı kimlik bilgileri, bu modül tarafından oluşturulan UUID üzerinden iletilir (IAR-1). Donanım kaynakları olarak JWT işlemleri yalnızca CPU gücü tüketir ancak Supabase Auth hizmeti tüm sunucu hizmetlerini çalıştırır, ek sunucu kaynaklarına ihtiyaç yoktur. İlerleme olarak tasarım yenidir. Supabase JS SDK v2+ ve Supabase Auth altyapısı kullanılacaktır.

4.1.2 WU-02: Profil ve Ağ Modülü

Amaç olarak bu modül, kullanıcı profillerinin takip/etkileşim ilişkilerinin oluşturulması, güncellenmesi ve sürdürülmesini içerir. Bu modül, kullanıcı adı, biyografi, yetenek etiketleri ve profil fotoğrafı gibi profil detaylarını yönetir. Takip ilişkileri PostgreSQL'deki 'follows' tablosu aracılığıyla yönetilir. Takipçi sayıları Supabase Veritabanı Tetikleyicileri kullanılarak korunur. Donanım kaynakları açısından profilinizi güncellemek için hafif REST API çağrılarına ihtiyaç vardır. Görsel yüklemeler Supabase Storage aracılığıyla yapılır. Geliştirme durumu olarak yeni tasarımdır. PostgreSQL yabancı anahtar yapıları ve Supabase Tetikleyici mekanizmaları kullanılacaktır.

4.1.3 WU-03: Feed ve İçerik Modülü

Bu modül, kullanıcıların metin, görsel veya kod blokları paylaşmasını ve takip ettikleri hesaplar tarafından yapılan gönderileri kronolojik sırayla görüntülemesini sağlamaktır. Gönderi oluşturma işlevi, kod bloklarını bir IDE bağlamına çekmek için bir sözdizimi vurgulama bileşeni ekleyecektir. Akış algoritması, kullanıcının takip ettiği hesaplar tarafından yazılan gönderileri, oluşturuldukları tarihe göre sıralayarak gösterir. Sayfalama, her isteğin ardından 20 gönderi sağlayan sonsuz kaydırma ile yönetilir. RLS, yalnızca gönderi sahibinin kendi gönderisini silebileceğini belirtir. Donanım kaynakları bağlamında veritabanı ile ilgili en karmaşık işlemler akış sorgularıdır ve bu işlemleri gerçekleştirdiğinizde bulabilmeniz için ilgili tüm sütunlara (user_id ve created_at sütunları) indeksler ekleyeceksiniz.

4.1.4 WU-04: Medya ve Etkileşim Modülü

Kullanım durumu olarak gönderilere görseller, beğeniler ve yorumlar ekleyin. İstemci tarafında görsel yüklemeler için maksimum 2 MB sınırı vardır ve bu, Supabase Storage'a yüklenmeden önce sıkıştırılır. PostgreSQL'de beğeni ve yorum işlemleri 'likes' ve 'comments' tablolarına yazılır. Aynı kullanıcının aynı gönderiyi tekrar tekrar beğenmemesi için Birleşik Birincil Anahtar mimarisi kullanılır. Supabase Realtime ile etkileşim sayaçları gerçek zamanlı olarak yenilenir. Donanım kaynakları açısından Supabase ücretsiz planındaki sınırlı depolama nedeniyle görsellerin boyutu sınırlıdır.

4.1.5 WU-05: Gerçek Zamanlı Mesajlaşma Modülü

Bu modül, kullanıcılar arasında gerçek zamanlı ve çift yönlü özel mesajlaşma altyapısı sağlamaktır. Mesaj istekleri üzerindeki trafik, WebSocket (Supabase Realtime / WSS) kullanılarak aktarılır. Daha sonra, sohbet geçmişini kalıcı olarak korumak için her mesajı veritabanındaki 'messages' tablosuna yazar. Ancak bağlantı kesilirse, sistem arka planda yeniden bağlanmaya çalışır ve yeniden bağlanırsa bekleyen mesajlar otomatik olarak yeniden gönderilir. Gönderenin kimliği (sender_id), Auth modülünden güvenli bir şekilde alınır ve mesaj kaydına gömülür. Donanım kaynakları olarak sohbet oturumu sırasında WebSocket bağlantıları açıktır. Tüm mevcut ağ bağlantıları da Supabase ücretsiz plan limitine dayanmaktadır.

4.1.6 WU-06: Ortak Altyapı

Bu modülde amaç, tüm modüllerde uygulanan kimlik doğrulama ara yazılımının merkezi yönetimi, hata yönetimi, RLS politikaları ve API yapılandırmasını ele almaktır. Her arka uç uç noktası, kimlik doğrulama gerektiren bir ara yazılım katmanından geçirilir. Geçersiz veya eksik belirteçlerle yapılan istekler, 401 Yetkisiz yanıtı ile yanıtlanır. Bu bileşen, modüller arası güvenlik ve tutarlılık standardını sağlar (IAR-4).

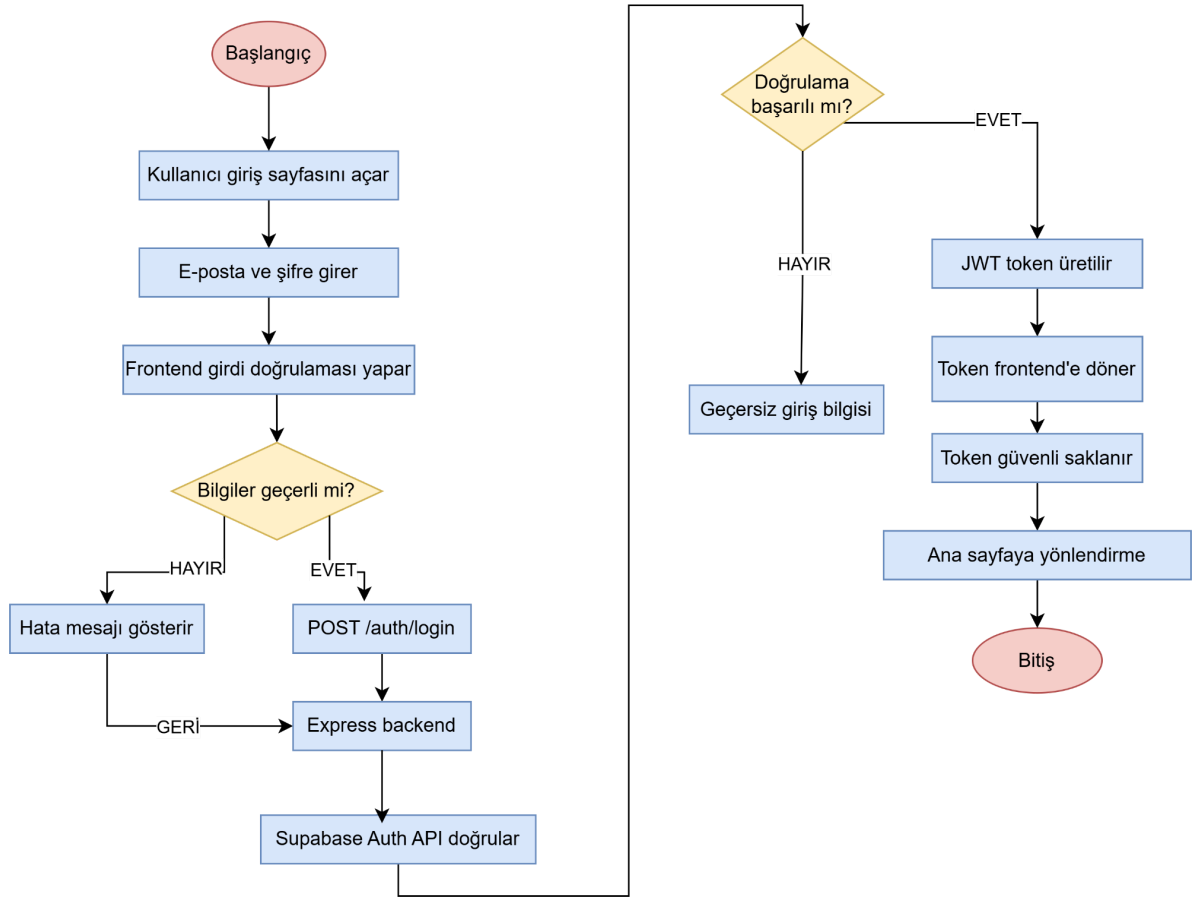
4.2 Genel Çalıştırma (Execution) Kavramı

Bu bölümde yazılım birimleri arasındaki çalışma zamanı etkileşimleri ve kontrol akışları açıklanmaktadır.

4.2.1 Kullanıcı Girişi Akış Diyagramı

Kullanıcı giriş süreci şu sırayla gerçekleşir:

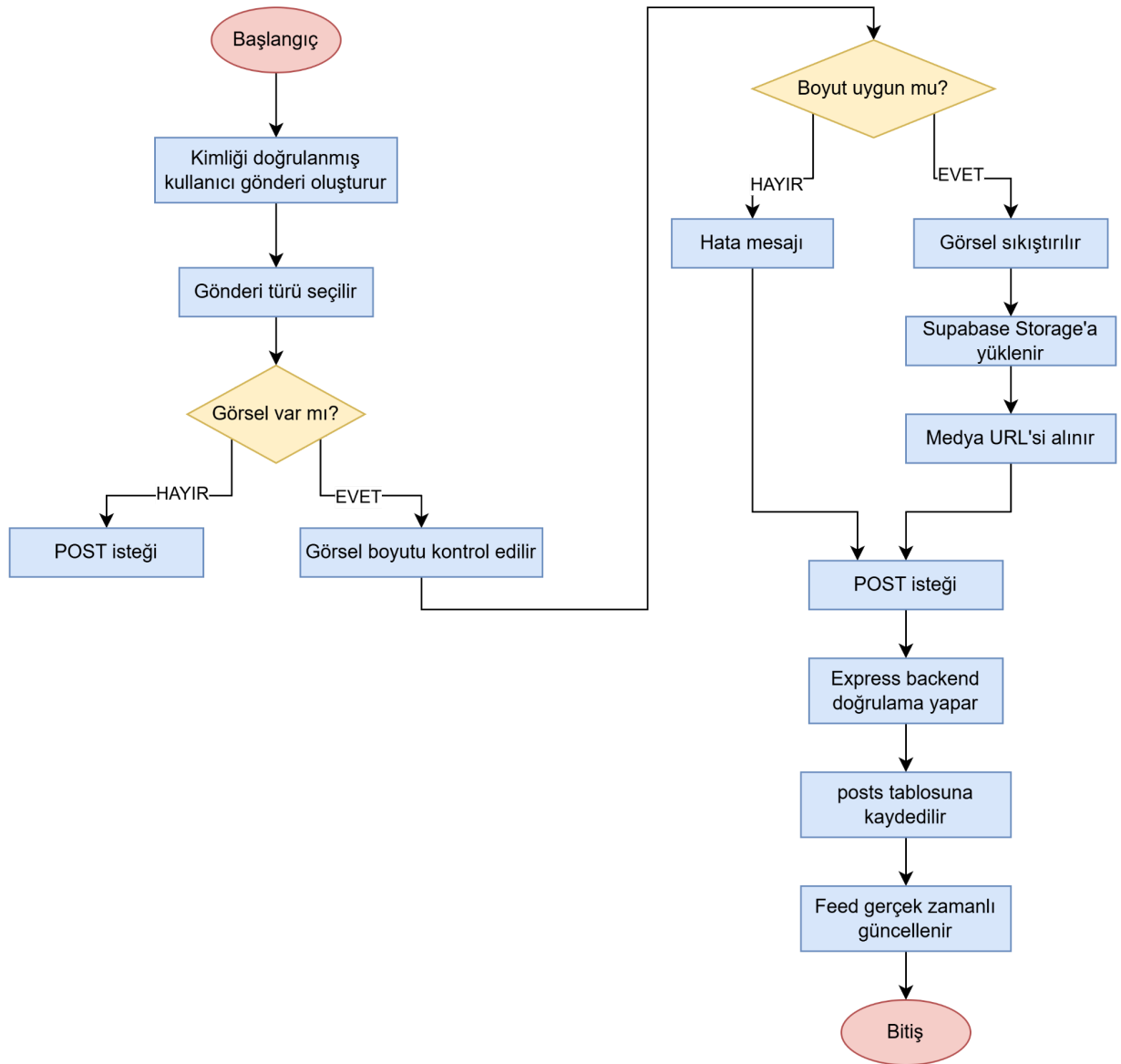
- Kullanıcı, tarayıcıda Giriş sayfasını ziyaret eder ve e-posta/şifre bilgilerini yazar.
- Girdi doğrulaması ön uçta yapılır ve bir POST /auth/login isteği gönderilir.
- Express.js arka ucu isteği kabul eder ve Supabase Auth API'sine gönderir.
- Supabase Auth kimlik bilgilerini doğrular ve bir JWT tokeni oluşturur.
- Token istemciye geri gönderilir; ön uç tokeni güvenli bir şekilde saklar.
- Tüm sonraki API isteklerinde, token Authorization başlığında gönderilir.
- JWT süresi dolduğunda, sistem kullanıcıyı otomatik olarak tekrar giriş yapmaya yönlendirir.



4.2.2 Gönderi Oluşturma Akış Diyagramı

Aşağıdaki sırayla, kullanıcı girişiyle oluşturulan gönderi yeni gönderiler oluşturur:

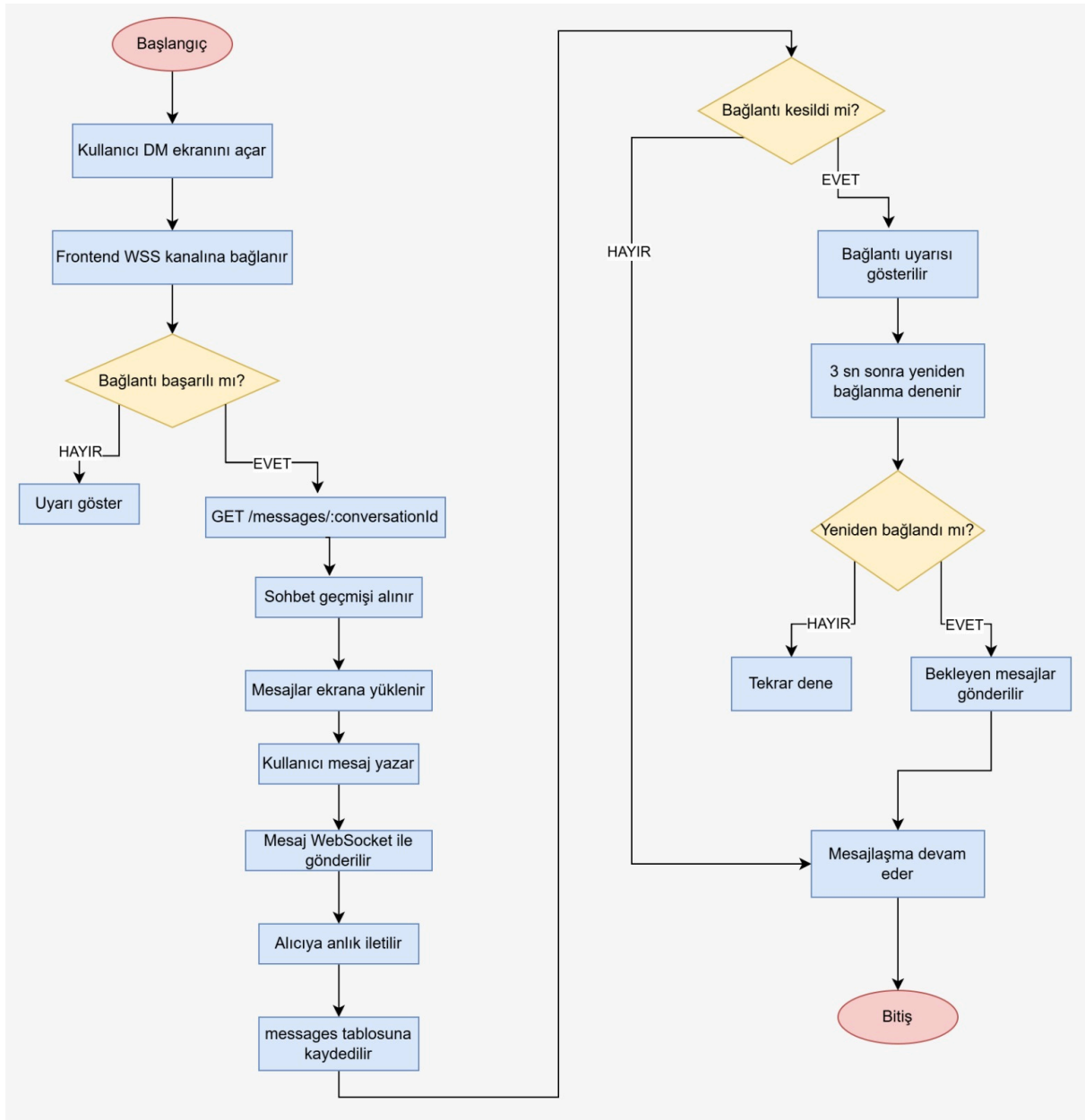
- Kimliği doğrulanmış bir kullanıcı gönderi oluşturma işlevini başlatır. Metin, resim veya kod bloğu türlerinden birini seçerler.
- Bir resim seçilirse, istemci tarafında bir boyut kontrolü yapılır (maks. 2 MB), ardından sıkıştırma işlemi gerçekleştirilir.
- Frontend, bir POST /posts isteği gönderir. Eğer bir resim varsa, önce Supabase Storage'a yüklenir ve döndürülen URL gönderi verilerine eklenir.
- Backend, gönderiyi RLS politikalarını kullanarak 'posts' tablosuna yazar.
- Akış modülü, yeni gönderiyi takipçilerin akışına gerçek zamanlı olarak ekler.



4.2.3 Gerçek Zamanlı Mesajlaşma Akış Diyagramı

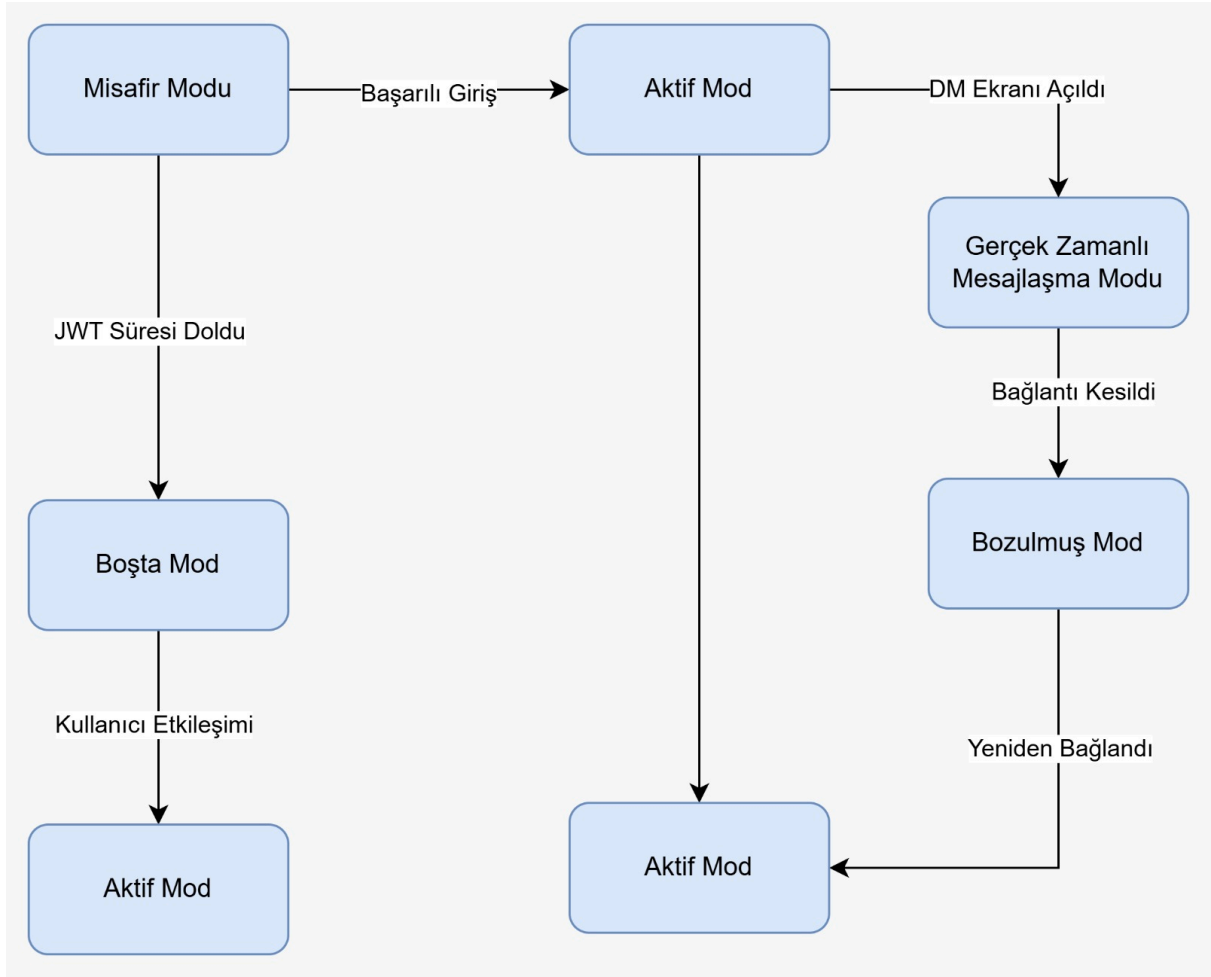
Gerçek zamanlı mesajlaşma zinciri şu şekilde gösterilmiştir

- DM ekranı açılacak ve ön yüz, Supabase Realtime aracılığıyla doğrudan WSS kanalına bağlanacaktır.
- Sohbet geçmişi, GET /messages/:conversationId uç noktasından alınacak ve ekrana yüklenecektir.
- Kullanıcı bir mesaj oluşturduğunda, bu mesaj anında WebSocket kanalı aracılığıyla diğer tarafa iletilir ve veritabanına yazılır.
- Bağlantı kaybolduğunda, kullanıcı arayüzü bir uyarı gösterir; her 3 saniyede bir yeniden bağlanmayı dener.
- Yeniden bağlandığında, bekleyen mesajlar otomatik olarak göndericiye gider.



4.2.4 Durum Geçiş Diyagramı

SRS'de tanımlanan sistem modlarına göre DevConnect'in durum geçişleri aşağıdaki gibidir:



4.3 Arayüz Tasarımı

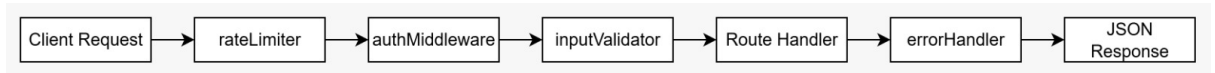
Bu bölüm, SRS Bölüm 1.3'te tanımlanan dış arayüzlerin tasarım detaylarını sunar.

Arayüz ID	Arayüz Adı	Tip	Veri Formatı	Bağlanan Birimler
IF-1	Kullanıcı Tarayıcısı	Kullanıcı-Sistem	HTTP/HTTPS, WebSocket	WU-01, WU-02, WU-03, WU-04, WU-05
IF-2	Supabase Auth API	Sistem-Servis	REST / HTTPS POST-GET	WU-01 (Auth Modülü)
IF-3	Supabase DB (PostgreSQL)	Sistem-Veritabanı	SQL Queries	Tüm WU modülleri
IF-4	Supabase Storage	Sistem-Servis	Multipart Form / HTTPS	WU-04 (Medya Modülü)
IF-5	Supabase Realtime / WebSocket	Sistem-Protokol	JSON (WSS)	WU-05 (Mesajlaşma)
IF-6	Express.js REST API	İç Sistem	JSON / HTTP	Frontend ↔ Tüm Backend Modüller

4.3.1 IF-6: Express.js REST API Endpoint Tasarımı

Backend API'nin temel endpoint tasarımı aşağıdaki tabloda özetlenmiştir:

Endpoint	HTTP Metodu	Açıklama	Auth Gerekli?
POST /auth/register	POST	Kullanıcı kaydı	Hayır
POST /auth/login	POST	Kullanıcı girişi, JWT döner	Hayır
POST /auth/logout	POST	Token geçersizleştirme	Evet
GET /users/:id	GET	Profil bilgisi getir	Evet
PUT /users/:id	PUT	Profil güncelle	Evet
POST /follows/:id	POST	Kullanıcıyı takip et	Evet
DELETE /follows/:id	DELETE	Takipten çık	Evet
GET /posts/feed	GET	Takip edilen kullanıcıların gönderileri	Evet
POST /posts	POST	Yeni gönderi oluştur	Evet
DELETE /posts/:id	DELETE	Gönderi sil (yalnızca sahip)	Evet
POST /posts/:id/like	POST	Beğeni ekle / kaldır	Evet
POST /posts/:id/comments	POST	Yorum ekle	Evet
GET /messages/:userId	GET	Sohbet geçmişi getir	Evet
POST /messages	POST	Mesaj gönder (DB'ye yaz)	Evet



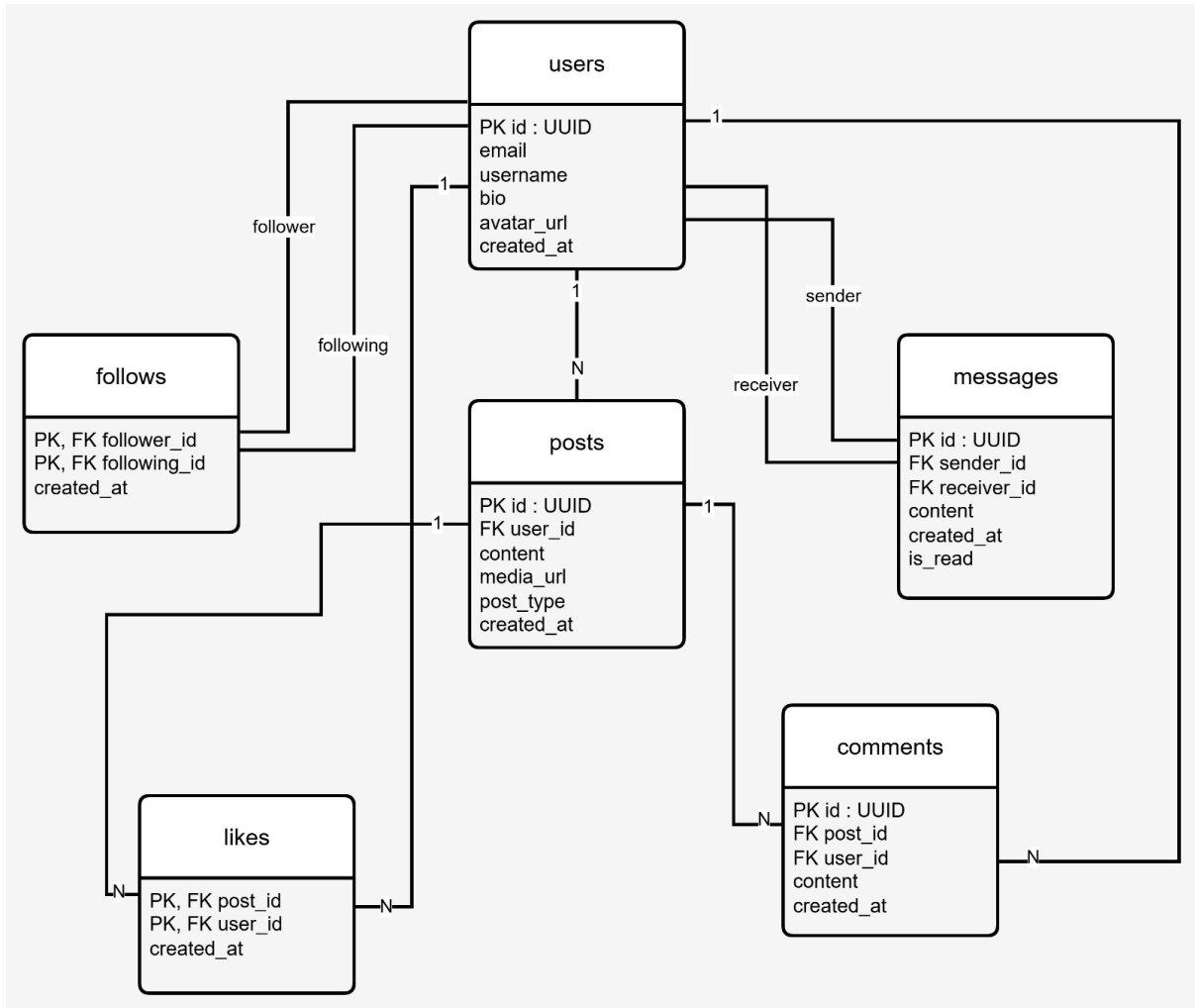
Tüm endpoint'ler TypeScript ile yazılmış Express Router modüllerine ayrılacaktır. Kimlik doğrulaması gerektiren her endpoint, JWT doğrulama middleware'inden geçirilecektir.

4.3.2 Veritabanı Şema Tasarımı

Sistem, SRS Bölüm 1.5'te tanımlanan veri gereksinimlerine uygun olarak aşağıdaki tablo yapısıyla inşa edilecektir:

Tablo	Birincil Anahtar	Temel Alanlar	İlişkiler
users	id (UUID PK)	email, username, bio, avatar_url, created_at	–

follows	follower_id + following_id (Composite PK)	follower_id (FK→users), following_id (FK→users), created_at	users tablosuna 2 FK
posts	id (UUID PK)	user_id (FK→users), content, media_url, post_type, created_at	users tablosuna FK
likes	post_id + user_id (Composite PK)	post_id (FK→posts), user_id (FK→users), created_at	posts ve users'a FK
comments	id (UUID PK)	post_id (FK→posts), user_id (FK→users), content, created_at	posts ve users'a FK
messages	id (UUID PK)	sender_id (FK→users), receiver_id (FK→users), content, created_at, is_read	users tablosuna 2 FK



5. YKE DETAYLI PLANI

Bu tartışma, her bir yazılım birimi (WU) için tasarım seçimleri, dayatılan kısıtlamalar, kullanılan dil, veri akışları ve veri yapısını detaylı bir şekilde incelemektedir.

5.1 WU-01: Auth Modülü Detaylı Planı

Programlama Dili: TypeScript (Express.js + Supabase JS SDK v2+).

Birim Tasarım Kararları:

Varsayılan olarak, JWT tokenları 1 saat sonra sona erer. Token süresi dolduğunda, kullanıcı otomatik olarak giriş sayfasına döner.

Şifre karma işlemi tamamen Supabase Auth altyapısına bağlıdır, başka bir kütüphane kullanılmamaktadır.

Birden fazla başarısız giriş denemesi için hız limiti uygulanacaktır (SAF-2).

RLS politikaları, başlangıçta kapalı olarak farklı tablolara uygulanacak ve test aşamasında kademeli olarak açılacaktır.

Sınırlar: E-posta adresi 254 karakteri aşmamalı; şifre en az 8 karakter olmalı; kullanıcı adı 50 karakteri aşmamalıdır.

Girdi/Çıktı Akışı:

- Girdi: { email: string, password: string }.
- Çıktı: { access_token: JWT, user_id: UUID, expires_at: Unix zaman damgası }.

Hata Durumları: 401 Yetkisiz (geçersiz kimlik bilgileri), 422 İşlenemez (eksik alan), 429 Çok Fazla İstek (hız limiti).

Mantık Akışı: İstek alınır → Girdi doğrulama adımları yapılır → Supabase Auth çağrılır → JWT oluşturulur, istemciye döndürülür → Sonraki istekler, Authorization başlığından tokeni okur ve doğrular.

5.2 WU-02: Profil ve Ağ Modülü Detaylı Planı

Programlama Dili: TypeScript (Express.js + Supabase JS SDK).

Birim Tasarım Kararları:

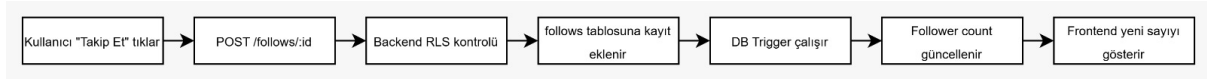
Kullanıcı kaydedildiğinde, 'users' tablosuna otomatik olarak boş bir profil satırı eklenir ve bu işlem Supabase Auth kancası kullanılarak yapılır.

Takipçi ve takip edilen sayıları, 'users' tablosundaki 'follower_count' ve 'following_count' alanları kullanılarak saklanacaktır; bu sayaçlar, veritabanı tetikleyicileri aracılığıyla otomatik olarak güncellenecektir.

Profil fotoğrafı Supabase Storage'a yüklenir, alınan URL 'users' tablosundaki 'avatar_url' alanına yazılır.

Sınırlamalar: Biyografi metni en fazla 300 karakterle sınırlıdır ve beceri etiketlerinin sayısı en fazla 10 ile sınırlıdır. Kullanıcı adını değiştirme yeteneği ilk sürümde entegre edilmeyecektir (OTH-2).

Takip etme durumumda (mantıksal öğelerin akışı)(takip-fonksiyonu): Kullanıcı 'Takip Et' düğmesine basar → Ön uç POST /follows/:id isteği gönderir → Arka uç RLS kontrolü yapar → 'follows' tablosuna kayıt eklenir → Veritabanı Tetikleyicisi takipçi sayaçlarını güncellemek için çağrılır → Ön uç yeni sayıyı gerçek zamanlı olarak gösterir.



5.3 WU-03: Feed ve İçerik Modülü Detaylı Planı

Programlama Dili: TypeScript. Frontend'de React Query, backend'de Express.js.

Birim Tasarım Kararları:

Akış sorgusu; kullanıcı 'follows' tablosundan takip ilişkisini arar ve takip edilen hesapların gönderilerini döndürür (IAR-2). Sonsuz kaydırma uygulaması, imleç tabanlı sayfalama kullanacaktır. Her istekte 20 gönderi alınır.

Hem Prism.js hem de highlight.js kütüphaneleri sözdizimi vurgulama için getirilecektir.

Arka uçta gönderi silme için RLS politikaları vardır. Silme işlemi yalnızca gönderi sahibine yapılabilir.

Algoritmalar: Gönderi listeleme sorgusu; gönderiler tablosunu takipler tablosu ile BİRLEŞTİR. Sorgu mantığı: 'Takip ettiklerimin gönderilerini oluşturulma tarihine göre azalan sırayla listele, son imleçten 20 tane al.'

Performans: Gönderiler.user_id ve gönderiler.created_at sütunlarına bileşik bir indeks eklenecektir. Bu, büyük veri kümelerinde sorgu süresini önemli ölçüde hızlandıracaktır.

5.4 WU-04: Medya ve Etkileşim Modülü Detaylı Planı

Programlama Dili: TypeScript (Ön uç: browser-image-compression kütüphanesi; Arka uç: Express.js, Depolama: Supabase Storage).

Birim Tasarım Kararları:

Görüntü yükleme hattı: Kullanıcı bir görüntü seçer → İstemci boyutu kontrol eder (2 MB'den büyükse reddeder) → Görüntüyü browser-image-compression ile sıkıştırır → Supabase Storage'a yükler → Döndürülen genel URL, POST /posts isteğine dahil edilir.

Beğeni işlemi doğası gereği idempotenttir: kullanıcı zaten beğendiyse, beğeniye geri çekebilir; değilse, beğeni ekler. Bu mantık tek bir uç nokta (POST /posts/:id/like) üzerinden yönetilir.

Beğeni/yorum sayacıları, Supabase Realtime ile gerçekleştiği anda güncellenir; böylece kullanıcı için sayfa yenilemeye gerek kalmaz (FR-4.6).

İstisna ve Hata Yönetimi: Supabase Storage yükleme hatası meydana geldiğinde, tüm gönderi oluşturma süreci sonlandırılır ve kullanıcıya bir hata mesajı görünür. İşlem atomik olacaktır, kısmi başarıları (görüntü yüklendi, veritabanı yazma başarısız oldu) durdurmak için.

5.5 WU-05: Gerçek Zamanlı Mesajlaşma Modülü Detaylı Planı

Programlama Dili: TypeScript (Supabase Realtime / WebSocket, ihtiyaç halinde Socket.IO fallback).

Birim Tasarım Kararları:

Supabase Realtime (WSS) ana iletişim protokolü olacaktır. Gerekirse Socket.IO yedekleme olarak kullanılabilir.

Yetkilendirilmiş WebSocket kanalı, JWT token ile yetkilendirilir. Yetkisiz kanallar, erişimi önlemek için otomatik olarak kapatılır.

Yeniden bağlantı stratejisi: Bağlantı kesildiğinde, yeniden bağlantı 3, 6, 12 saniye aralıklarla denenecektir (üstel geri çekilme).

Mesaj geçmişi, sohbet ekranı açıldığında GET /messages/:userId uç noktasından çekilir ve kronolojik olarak görüntülenir. Yeni mesajlar, WebSocket üzerinden gerçek zamanlı olarak eklenir.

'is_read' alanı, alıcı mesajı okuduğunda okuma durumunu izlemek için değişir.

Veri Formatı (WebSocket): Her mesaj, JSON yapısı kullanılarak iletilir: { sender_id: UUID, receiver_id: UUID, content: string, timestamp: ISO8601 }.

Zamanlama: Mesaj teslimatındaki gecikme için zaman hedefi < 500 ms'dir (gereklilik IF-5). Bu hedef, Supabase Realtime altyapısının performans yeteneklerine bağlıdır.

5.6 WU-06: Ortak Altyapı Detaylı Planı

Programlama Dili: TypeScript (Express.js Middleware).

Birim Tasarım Kararları:

authMiddleware – Bu ara yazılım (her korunan uç noktaya eklenir) JWT tokeni Authorization başlığından çıkarır, doğrular ve kullanıcı bilgilerini req.user nesnesine ekler.

errorHandler: Küresel hata yakalama ara yazılımı. Tüm modüllerden atılan hatalar burada merkezi olarak ele alınır ve standart bir JSON formatında yanıtlanır.

rateLimiter: İstekler, express-rate-limit kütüphanesi kullanılarak IP ile sınırlanır. Giriş uç noktaları için dakikada maksimum 10 istek sınırı uygulanır.

inputValidator: Form girdileri express-validator aracılığıyla doğrulanır ve ayrıca temizleme işlemleri gerçekleştirilir.

Ara Yazılım Yürütme Sırası: İstek → rateLimiter → authMiddleware → inputValidator → Rota İşleyici → errorHandler.

6. GEREKSİNİMLERİN İZLENEBİLİRLİĞİ

Bu bölüm, SRS'deki gereksinimlerin tasarım dokümanındaki yazılım birimleriyle çift yönlü izlenebilirliğini sağlamaktadır.

6.1 SRS Gereksinimlerinden Yazılım Birimlerine İzlenebilirlik

SRS Gereksinim ID	Gereksinim Açıklaması (Özet)	İlgili WU Birimi(leri)
FR-1.1 – FR-1.7	Kullanıcı kaydı, girişi, JWT yönetimi, RLS, çıkış	WU-01, WU-06
FR-2.1 – FR-2.6	Profil yönetimi, takip/takipten çıkma	WU-02
FR-3.1 – FR-3.5	Gönderi oluşturma (metin/kod/görsel), feed algoritması, silme	WU-03
FR-4.1 – FR-4.6	Görsel yükleme, beğeni, yorum, gerçek zamanlı sayaçlar	WU-04
FR-5.1 – FR-5.6	Özel mesajlaşma, WebSocket, mesaj geçmişi, yeniden bağlanma	WU-05
IAR-1	UUID ile kullanıcı tanımlama (tüm modüller)	WU-01, WU-06
IAR-2	Feed'de takip filtresi	WU-03
IAR-3	Medya URL'inin gönderi ile ilişkilendirilmesi	WU-03, WU-04
IAR-4	Tüm endpoint'lerde kimlik doğrulaması zorunluluğu	WU-06
SEC-1 – SEC-6	HTTPS, şifre hashleme, RLS, SQL Injection/XSS önlemleri	WU-01, WU-06
CONF-1 – CONF-3	JWT süresi, ortam değişkenleri, görsel boyut limiti	WU-01, WU-04

6.2 Yazılım Birimlerinden SRS Gereksinimlerine İzlenebilirlik

WU Birimi	Birim Adı	Karşılanan SRS Gereksinimleri
WU-01	Auth Modülü	FR-1.1, FR-1.2, FR-1.3, FR-1.4, FR-1.5, FR-1.6, FR-1.7, SEC-1, SEC-2, SEC-3, CONF-1
WU-02	Profil ve Ağ Modülü	FR-2.1, FR-2.2, FR-2.3, FR-2.4, FR-2.5, FR-2.6
WU-03	Feed ve İçerik Modülü	FR-3.1, FR-3.2, FR-3.3, FR-3.4, FR-3.5, IAR-2, IAR-3
WU-04	Medya ve Etkileşim Modülü	FR-4.1, FR-4.2, FR-4.3, FR-4.4, FR-4.5, FR-4.6, CONF-3
WU-05	Gerçek Zamanlı Mesajlaşma Modülü	FR-5.1, FR-5.2, FR-5.3, FR-5.4, FR-5.5, FR-5.6, SEC-5
WU-06	Ortak Altyapı	IAR-1, IAR-4, SEC-3, SEC-4, SEC-6, SAF-1, SAF-2, CONF-2

7. NOTLAR

Bu bölüm, dokümanın anlaşılmasına yardımcı olan genel bilgiler, kısaltmalar ve terim tanımlarını içermektedir.

7.1 Kısaltmalar ve Tanımlar

Kısaltma / Terim	Açıklama
SDD	Software Design Document – Yazılım Tasarım Dokümanı
SRS	Software Requirements Specification – Yazılım Gereksinim Belirtimi
SPMP	Software Project Management Plan – Yazılım Proje Yönetim Planı
YKE	Yazılım Konfigürasyon Elemanı (Software Configuration Item)
WU	Yazılım Birimi (Software Unit) – Bu dokümanda kullanılan modül tanımlayıcısı
JWT	JSON Web Token – Kimlik doğrulama için kullanılan token standardı
RLS	Row Level Security – Supabase/PostgreSQL satır bazlı erişim kontrolü
REST	Representational State Transfer – API mimarisi
DM	Direct Message – Özel/Doğrudan Mesaj

WSS	WebSocket Secure – Şifreli WebSocket protokolü
HTTPS	Hypertext Transfer Protocol Secure – Şifreli HTTP protokolü
UUID	Universally Unique Identifier – Evrensel benzersiz tanımlayıcı
FK	Foreign Key – Yabancı anahtar (veritabanı ilişkisi)
PK	Primary Key – Birincil anahtar
MVP	Minimum Viable Product – Minimum Uygulanabilir Ürün
SSR	Server-Side Rendering – Sunucu taraflı sayfa oluşturma
CDN	Content Delivery Network – İçerik Dağıtım Ağı
API	Application Programming Interface – Uygulama Programlama Arayüzü
DC	DevConnect – Geliştirilen projenin kısaltması
FR	Functional Requirement – Fonksiyonel Gereksinim
IAR	Internal Architecture Requirement – Dahili Mimari Gereksinim
SEC	Security Requirement – Güvenlik Gereksinimi
SAF	Safety Requirement – Emniyet Gereksinimi
CONF	Configuration Requirement – Yapılandırma Gereksinimi

7.2 Genel Notlar

Bu doküman, BM314 Yazılım Mühendisliği dersi kapsamında üçüncü temel teslimat olan SDD aşamasına karşılık gelmektedir. SPMP (11 Mart 2026) ve SRS (01 Nisan 2026) belgeleriyle birlikte değerlendirilmelidir.

Tasarım kararlarının bir kısmı Supabase ücretsiz plan sınırlamalarını göz önünde bulundurarak alınmıştır; bu nedenle bazı tasarımlar (örn. görsel boyut kısıtlaması, Storage seçimi) proje ölçeğiyle orantılıdır.

8. EKLER

Ek A – Teknoloji Yığını Özeti

Katman	Teknoloji	Sürüm	Kullanım Amacı
Frontend	React	v18+	UI bileşen kütüphanesi
Frontend	Next.js	v14+	SSR/SSG React çatısı
Frontend	Tailwind CSS	v3+	Utility-first CSS
Backend	Node.js	v18+	Çalışma ortamı
Backend	Express.js	v4+	REST API çatısı
Backend	TypeScript	v5+	Tip güvenli geliştirme
Veritabanı	PostgreSQL (Supabase)	–	İlişkisel veri
Depolama	Supabase Storage	–	Medya dosyaları
Auth	Supabase Auth	–	JWT + RLS
Gerçek Zamanlı	Supabase Realtime / Socket.IO	v4+	WebSocket iletişim
Dağıtım	Vercel	–	Frontend deployment
Sürüm Kontrolü	Git + GitHub	–	Kaynak kod yönetimi